

Sets - creation, adding and removing elements.

A set in Python is an unordered collection of **unique** elements. Sets are mutable, do not allow duplicate values and support mathematical set operations.

1. Sets - Creation

We can create sets using curly braces {} or the set() constructor.

```
a = {1, 2, 3, 4}      # using curly braces
b = set([1, 2, 3, 4]) # using set() constructor
c = set('hello')      # set of characters (duplicates removed)
d = set()             # empty set
```

Output :

```
a → {1, 2, 3, 4}
b → {1, 2, 3, 4}
c → {'h', 'e', 'l', 'l', 'o'}
d → set()
```

2. Adding Elements to a Set

We can add elements to a set using the add() method.

```
s = {10, 20}
s.add(30)      # adds single element
s.add(20)      # 20 already exists, no change
s.update([40, 50]) # adds multiple elements
```

Output :

```
s → {10, 20, 30, 40, 50}
```

3. Removing Elements from a Set

We can remove elements from a set using different methods.

```
s = {10, 20, 30, 40, 50}
s.remove(20)    # removes 20, raises KeyError if not found
s.discard(100)  # tries to remove 100, does not raise error
popped = s.pop() # removes and returns a random element
s.clear()       # removes all elements
```

Output / Result :

```
After remove(20) → {10, 30, 40, 50}
After discard(100) → {10, 30, 40, 50}
popped → (random element from set)
After clear() → set()
```

Key Points

- Sets store only unique elements.
- Order is not preserved.
- Use add() / update() to add elements.
- Use remove() / discard() / pop() / clear() to remove elements.

Sets - creation

A set in Python is an unordered collection of **unique** elements. Sets are mutable, do not allow duplicate values and support mathematical set operations.

1. Creating Sets

We can create sets using curly braces {} or the set() constructor.

Example :

```
a = {1, 2, 3, 4}          # using curly braces
b = set([1, 2, 3, 4])     # using set() constructor
c = set('hello')          # set of characters (duplicates removed)
d = set()                 # empty set
```

Output :

```
a → {1, 2, 3, 4}
b → {1, 2, 3, 4}
c → {'h', 'e', 'l', 'l', 'o'}
d → set()
```

Explanation :

- `a = {1, 2, 3, 4}` → Set created using curly braces.
- `b = set([1, 2, 3, 4])` → Set created using set() constructor.
- `c = set('hello')` → Characters of the string are added as unique elements. Duplicates are removed.
- `d = set()` → Creates an empty set.

Key Points

- Sets are unordered and do not support indexing.
- Duplicate values are automatically removed.
- Elements in a set must be immutable (e.g., numbers, strings, tuples).
- Use set() to create an empty set.

Sets - adding

We can add elements to a set using the `add()` method. Since sets are unordered and do not allow duplicates, only unique elements are added.

1. Using `add()` method

The `add()` method adds a single element to the set.

Example :

```
s = {10, 20, 30}    # initial set
s.add(40)            # adds 40
s.add(20)            # 20 already exists, no change
s.add('hello')       # adds 'hello'
print(s)             # output the set
```

Output :

{10, 20, 30, 40, 'hello'}

2. Adding Multiple Elements

We can add multiple elements using the `update()` method (using another set, list, tuple, or string).

Example :

```
s = {1, 2}          # initial set
s.update([3, 4, 5])  # add multiple elements from a list
s.update({5, 6, 7})  # add multiple elements from another set
s.update('ab')       # add characters of a string
print(s)
```

Output :

{1, 2, 3, 4, 5, 6, 7, 'a', 'b'}

Key Points

- `add()` adds a single element to the set.
- If the element already exists, the set remains unchanged.
- `update()` adds multiple elements from an iterable (list, set, tuple, string, etc.).
- Only unique elements are stored in a set.

Sets - removing elements.

We can remove elements from a set using different methods.

If an element is not found, some methods raise an error while others do nothing.

1. Using `remove()` method

The `remove()` method removes a specific element.

If the element is not present, it raises a `KeyError`.

Example :

```
s = {10, 20, 30, 40, 50}    # initial set
s.remove(30)                 # removes 30
s.remove(60)                 # raises KeyError (60 not found)
print(s)                     # output the set
```

Output / Result :

After `remove(30)` → {10, 20, 40, 50}

After `remove(60)` → `KeyError`

2. Using `discard()` method

The `discard()` method removes a specific element.

If the element is not present, no error occurs.

Example :

```
s = {10, 20, 30, 40, 50}    # initial set
s.discard(20)                # removes 20
s.discard(60)                # no error (60 not found)
print(s)                     # output the set
```

Output :

After `discard(20)` → {10, 30, 40, 50}

After `discard(60)` → {10, 30, 40, 50}
(no change)

3. Using `pop()` method

The `pop()` method removes and returns a random element.

If the set is empty, it raises a `KeyError`.

Example :

```
s = {10, 20, 30}            # initial set
x = s.pop()                  # removes and returns a random element
print("Removed:", x)         # print removed element
print("Remaining set:", s)    # print remaining set
```

Output / Result :

Removed: 20 (random element)

Remaining set: {10, 30}

4. Using `clear()` method

The `clear()` method removes all elements from the set.

Example :

```
s = {1, 2, 3, 4}            # initial set
s.clear()                    # removes all elements
print(s)                     # output the set
```

Output :

`set()` (empty set)

Key Points

- `remove(x)` → removes `x`, raises `KeyError` if `x` not found.
- `discard(x)` → removes `x`, does not raise error if `x` not found.
- `pop()` → removes and returns a random element.
- `clear()` → removes all elements from the set.